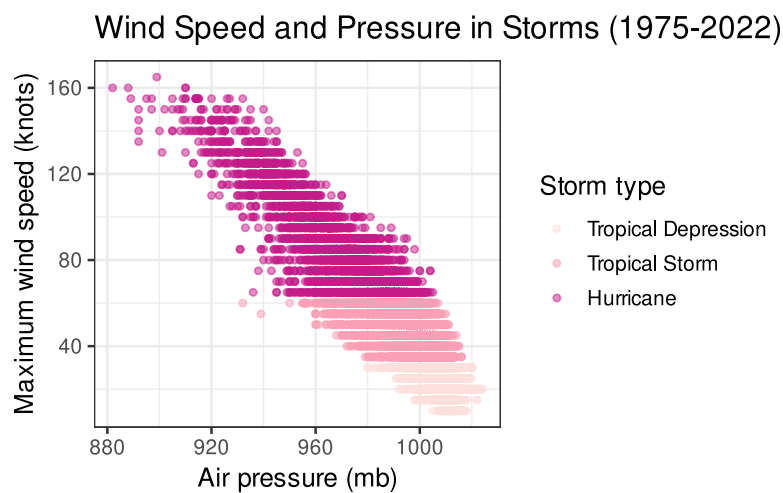


Explanatory Data Analysis

All of the plots in this problem set are meant to focus on a particular story to tell your a particular audience. As you craft them, think carefully about the labels, themes, aesthetic settings, and the manner in which it facilitates comparisons.

1. Provide the code that will create this plot from the `storms` data set inside the `dplyr` package. Here, each dot is a single storm. A few notes:
 - You will need to create a new column indicating storm type. For that you'll need to learn how that determination is made and then add it to the data frame. The `case_when()` function is a helpful shortcut similar to `ifelse()`.
 - The color palette belows to a set of palettes called ColorBrewer. You can read more about them here: https://ggplot2.tidyverse.org/reference/scale_brewer.html.



2. Are there any changes that you'd make to the plot above to make it easier to read or clarify what it is showing?

3. Remake your plot using a very specific visual theme: the botanical paintings of the US Department of Agriculture. They have been wrapped up into a ggplot2 theme inside the `ggpomological` package: <https://github.com/gadenbuie/ggpomological>. Read about how to use the package and write below the additional layers that you added to your plot above (for a challenge, see if you can get the handwriting font to show up).

4. The `present` data frame stores data from the US Census about the number of boys and girls born each year between 1940 and 2013. It is stored in the `stat20data` package, which can be installed from GitHub using the `devtools` package.

```
# install.packages("devtools")
# devtools::install_github("https://github.com/stat20/stat20data")
library(stat20data)
slice(present, 1:5)
```

Make a plot to visualize the trend over time in the proportion of births that are girls. Put the annual variability into context by ensuring the viewer can see the full range of possible values that the proportion could take. Allow the comparison to a baseline of .5 by drawing a horizontal line at that value (see `geom_hline()`).

5. (Challenge) Some of the most effective plots on the web don't come from R or Python, but from dedicated visualization libraries written in JavaScript. That is the case for the EV registration plot made by the Department of Energy (<https://afdc.energy.gov/data/10962>), which uses a library called **Highcharts**.

While you could craft the plot directly using JavaScript, a member of the R community has written a package that allows you to make them directly from R: you write R code, which the package turns into Javascript, which Highcharts then renders as a plot. Read how to use the package on its documentation site: <https://github.com/jbkunst/highcharter/>.

Your task is to recreate the EV Highcharts plot exactly using the data found in an excel spreadsheet on the DoE website. You can export the excel spreadsheet as a csv file, as we did in class, or you can experiment with the `readxl` package to read it into R directly.

Joins

Coming soon!